Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 4

Disciplina: Tehnici de programare

Tema: ” Cunostinta cu Python ”

A efectuat : st. gr.SI – 263 Boicu Ecaterina

A verificat : asistent universitar Reițman Patricia

Chișinău – 2026

SARCINA LUCRĂRII

Sarcina a fost încadrarea sa în bazele limbajului Python . Am avut de făcut cu primele comenzi din Python , prin intermediul jocurilor si site-urilor de predare , care presupun ușurarea incadrarii pentru incepatori.

Etapele Lucrării

1. *Accesarea site-urilor , drept sursă*
2. *Implementarea sarcinilor de pe site-ul sursa*
3. *Drept sursă dovada - capturarea ecranului (screenshot) cu rezultatele obtinute in terminal.*

1.Accesarea site-urilor

Primul site a fost **:**

https://www.codedex.io/python

Al doilea **:**

https://www.learnpython.org/

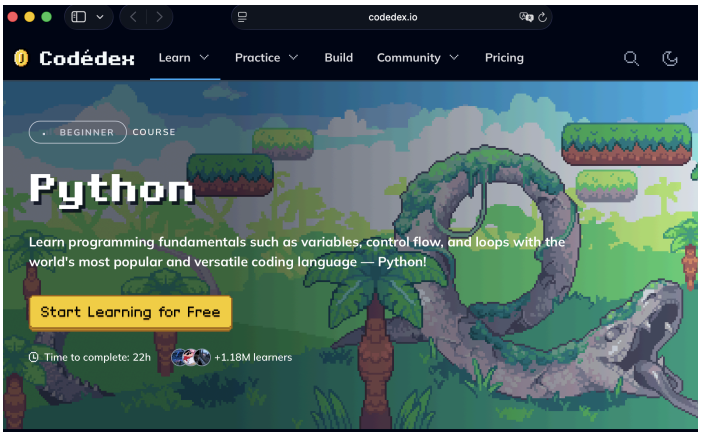


fig.1

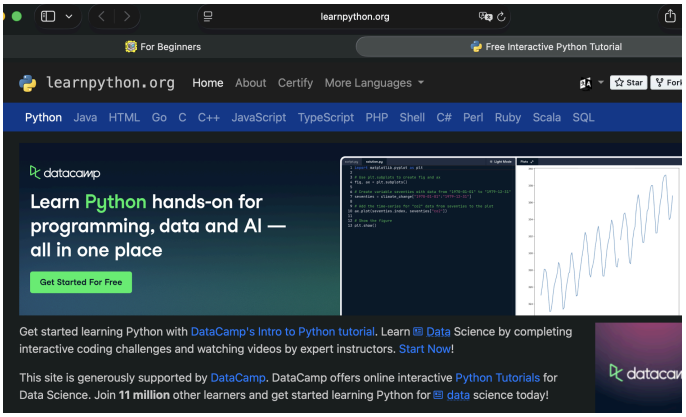


fig.2

2.Implementarea Sarcinilor

3.Sursa Dovada

Partea I : Codedex.io

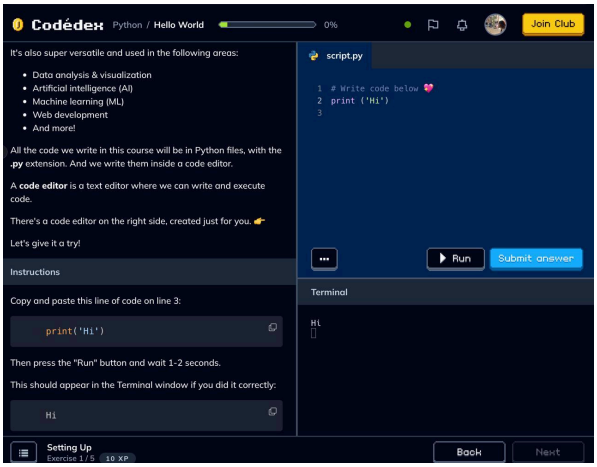


fig.1.1

Imaginea surprinde un ecran de laptop pe care este deschisă o lecție introductivă de programare în Python pe platforma Codédex, unde se execută cu succes comanda de afișare print('Hi!') în fereastra de terminal.

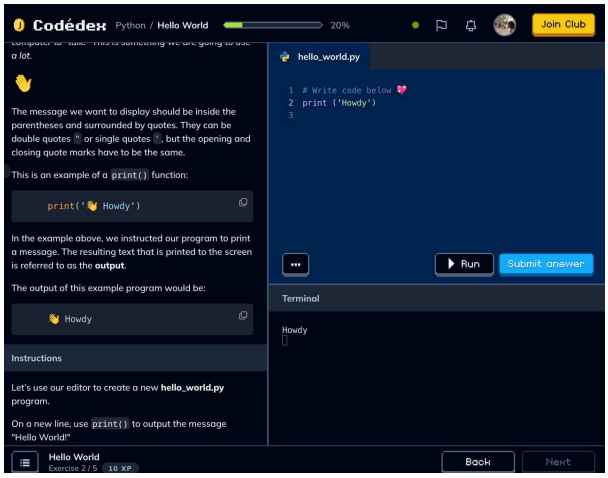


fig.1.2

Această etapă ilustrează utilizarea funcției print(), explicând că mesajul afișat trebuie plasat între paranteze și încadrat obligatoriu de același tip de ghilimele, fie simple, fie duble.

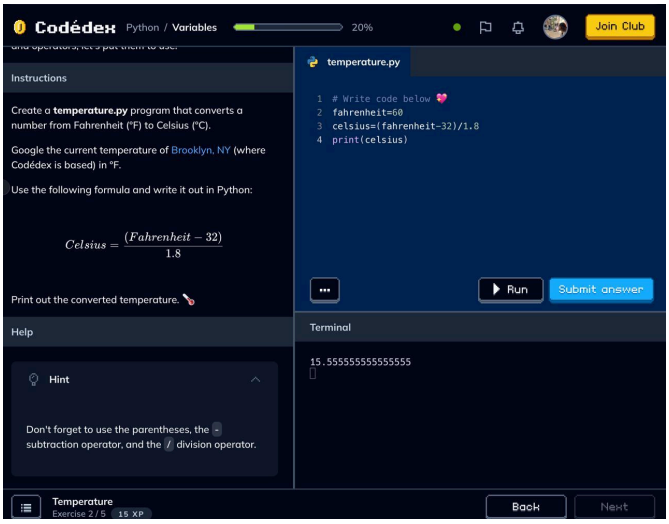


fig.1.3

Această etapă vizează exersarea utilizării variabilelor și a operatorilor aritmetici**(+;-;/;%;*)**prin crearea unui script de conversie a temperaturii din Fahrenheit în Celsius, **rezultatul obținut în terminal fiind 15.555555555555555**, ceea ce reprezintă valoarea echivalentă în grade Celsius pentru 60°F.

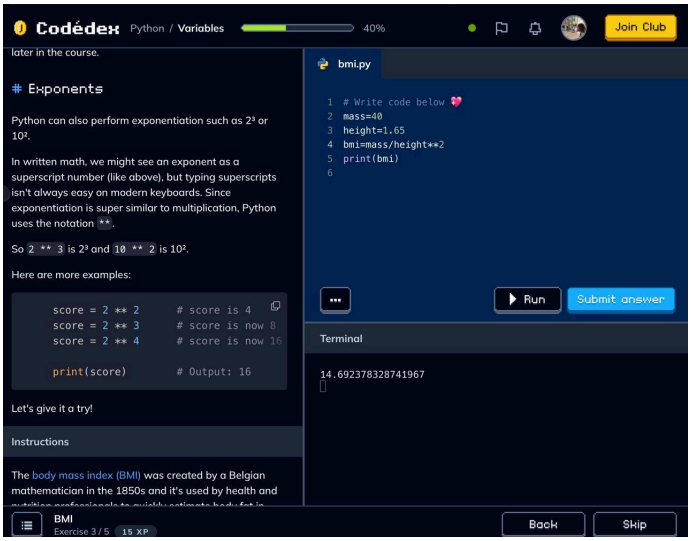
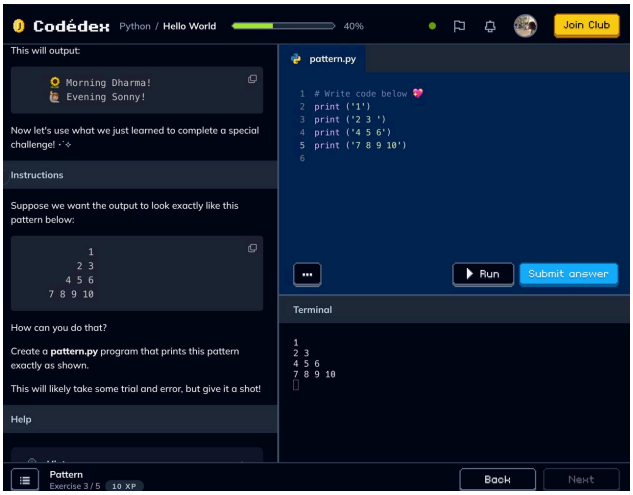
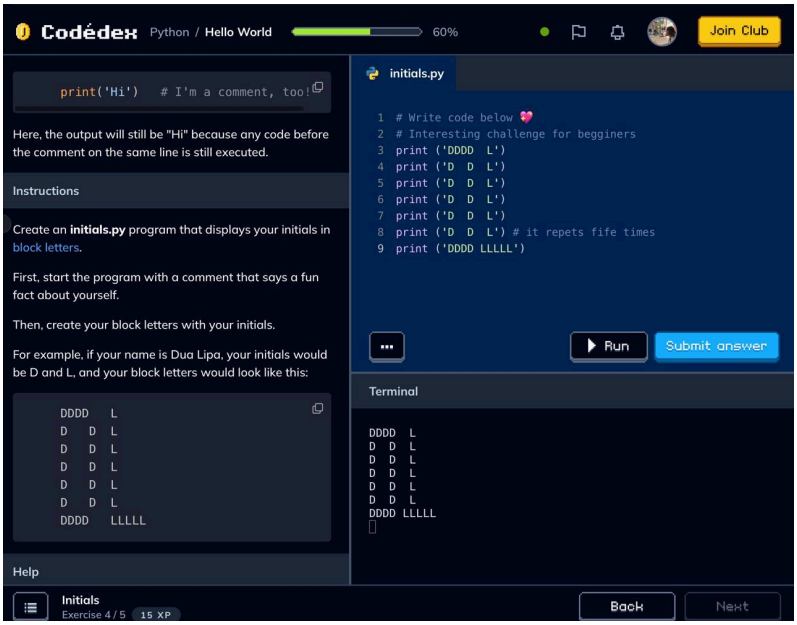
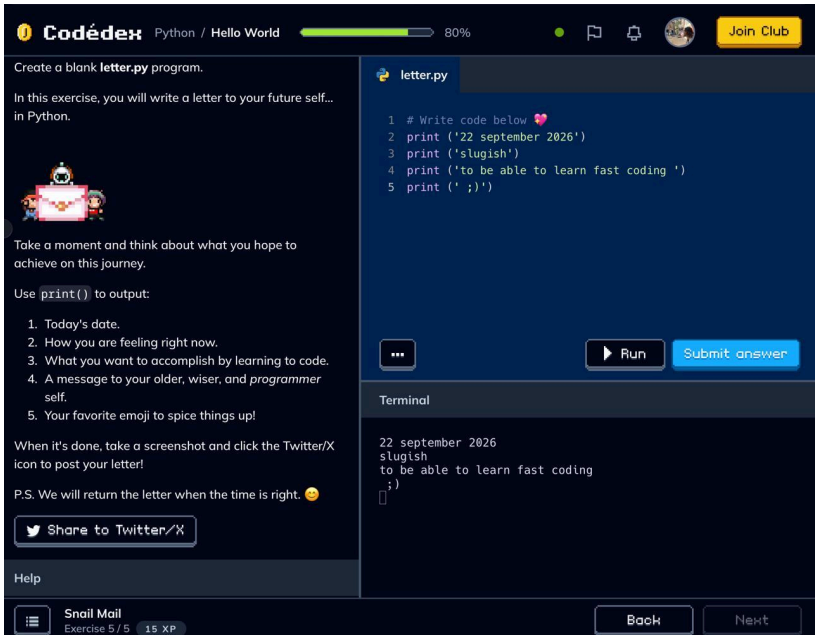


fig.1.4

Această etapă ilustrează utilizarea operatorului de exponențiere ** (două asteriscuri), care în limbajul Python îndeplinește **rolul de ridicare a unui număr la o putere** — înlocuind notația matematică cu superscript pentru a calcula înălțimea la pătrat (height**2) — și **returnează în terminal valoarea calculată a indicelui de masă corporală de 14.692378328741967.**







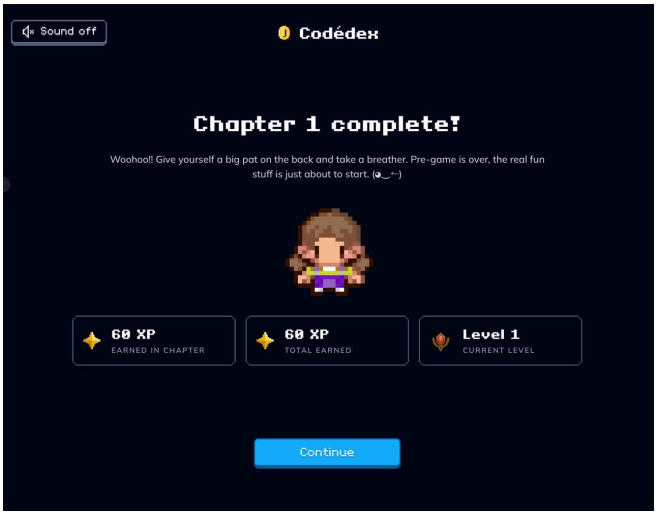


fig.1.5

1. Initials.py: S-a folosit funcția print() și comentarii pentru afișarea inițialelor în format bloc.

2. Pattern.py: S-a utilizat funcția print() pentru generarea unui șablon numeric în trepte.

3. Letter.py: S-a folosit funcția print() pentru redactarea unei scrisori personalizate către viitorul sine.

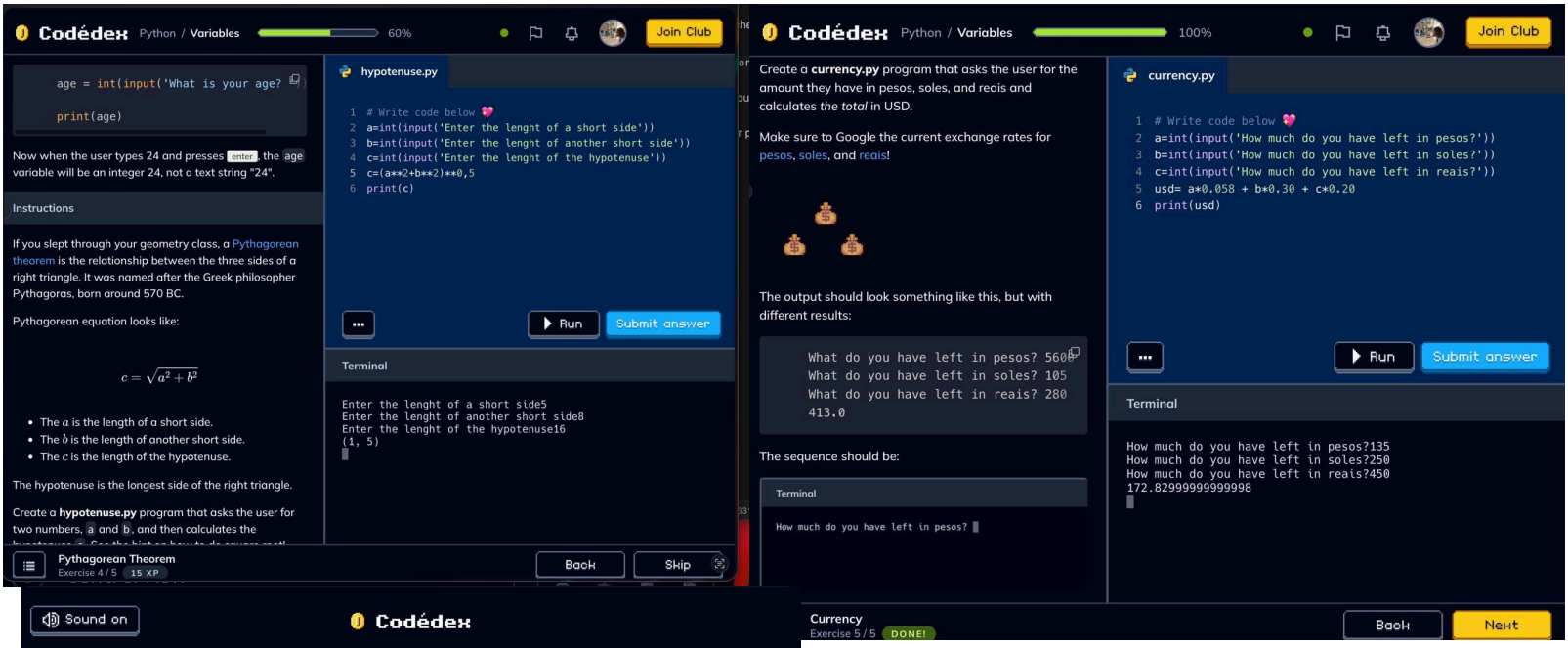


fig.1.6

Prima imagine (hypotenuse.py): Etapa ilustrează utilizarea funcțiilor int() și input() împreună cu operatorul de exponențiere **pentru a calcula ipotenuza unui triunghi dreptunghic prin Teorema lui Pitagora**, afișând rezultatul numeric în terminal și confirmând eficiența platformei Codédex în învățarea practică a programării.
A doua imagine (currency.py): Această etapă prezintă implementarea unui program de conversie valutară folosind funcții de citire interactivă și operatori aritmetici pentru a transforma diverse monede în dolari americani, afișând suma totală în terminal și evidențiind modul în care Codédex facilitează aplicarea noțiunilor teoretice în scenarii reale.

Concluzie

Platforma Codédex oferă în format de joc , învățare extrem de intuitiva , captivanta și orientata spre practică, reușind să transforme concepte teoretice fundamentale din Python — precum sintaxa de bază, variabilele, operatorii aritmetici și interacțiunea cu utilizatorul — în proiecte reale și atractive care consolidează eficient competențele de programare.

Partea II : LearnPython

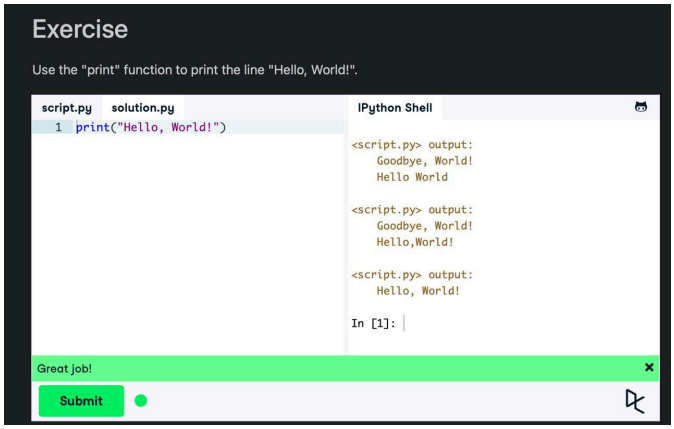


fig.1.7

Pentru a îndeplini sarcina de a folosi funcția print în vederea afișării mesajului „Hello, World!”, s-a utilizat sintaxa corespunzătoare a funcției împreună cu un șir de caractere încadrat de ghilimele, respectând cerințele de implementare, obținându-se în IPython Shell execuția cu succes a scriptului și afișarea promptului pentru următoarea comandă.

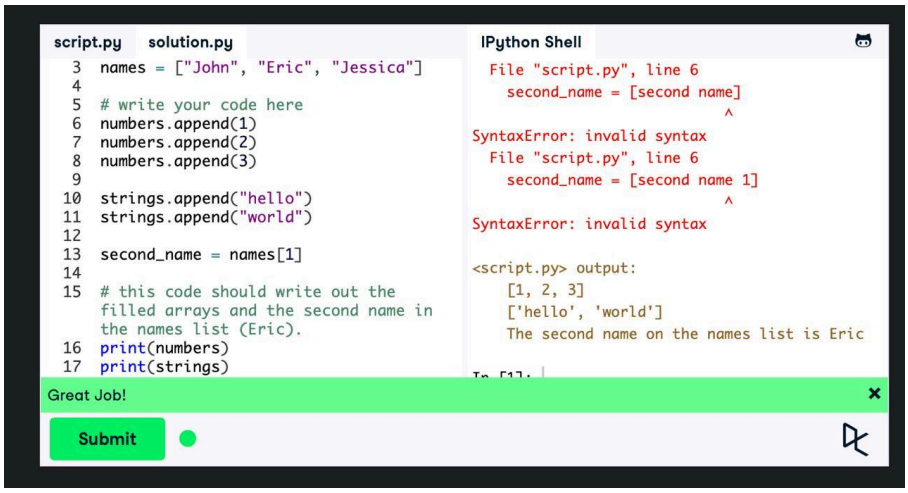


fig.1.8

Pentru a îndeplini sarcina esențială de prelucrare a listelor și extragere a celui de-al doilea element (names[1]), s-au utilizat metoda append() pentru adăugarea de elemente și funcția print() pentru afișare, respectând cerințele de sintaxă ale codului, iar în IPython Shell, după corectarea erorilor inițiale de sintaxă, s-a obținut afișarea cu succes a listelor numerice și textuale, precum și identificarea corectă a numelui Eric.

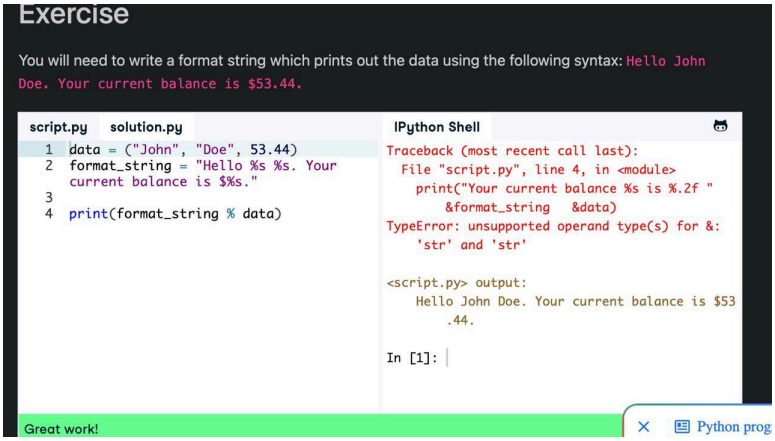


fig.1.9

Pentru a îndeplini exercițiul de a scrie un șir formatat care afișează datele sub forma „Hello John Doe. Your current balance is $53.44”, s-au utilizat o structură de date de tip tuplă (data), un șir cu marcatori %s și funcția print(), respectând cerințele de sintaxă ale limbajului, iar în IPython Shell,s-a obținut cu succes afișarea textului dorit.

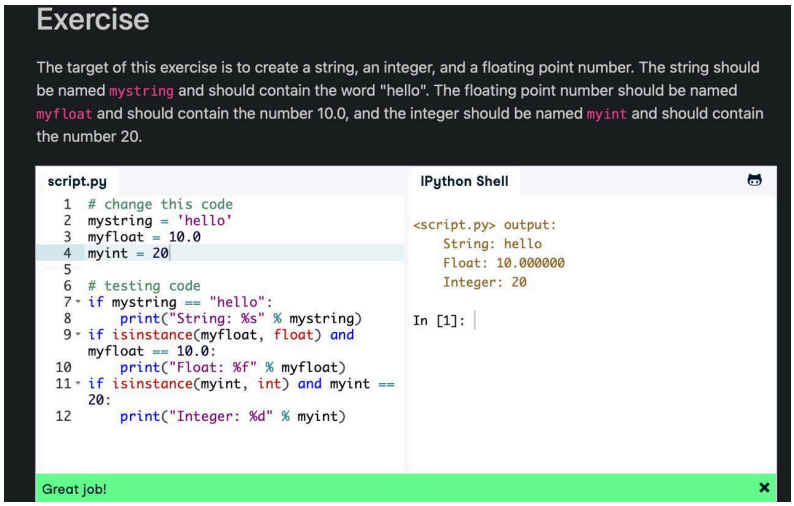


fig.1.10

Pentru a îndeplini sarcina de a crea și verifica variabile de tip șir (mystring), număr real (myfloat) și număr întreg (myint), s-au folosit instrucțiuni condiționale, funcția isinstance() și funcția print(), obținându-se în IPython Shell afișarea corectă a valorilor pentru fiecare tip de dată în parte.

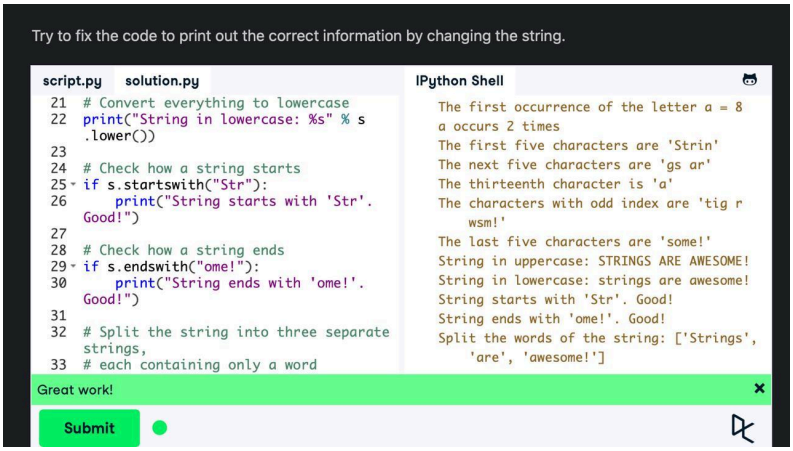


fig.1.11

Pentru a manipula şi a verifica un şir de caractere, s-au folosit metodele .lower(), .upper(), .startswith(), .endswith() şi .split(), obținându-se în IPython Shell conversiile textului, mesajele de validare şi lista de cuvinte separate ['Strings', 'are', 'awesome!'].

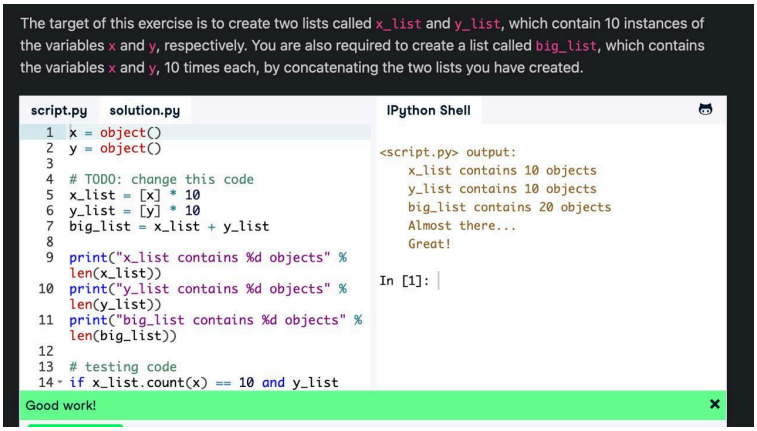


fig.1.12

Pentru a genera liste multiplicate pe baza unor obiecte şi a le unii, s-au utilizat operatorul de multiplicare (*), operatorul de adunare (+), funcția len() pentru determinarea dimensiunii şi marcatorul %d, obținându-se în IPython Shell afişarea cu succes a rezultatelor: 10 obiecte pentru x_list, 10 pentru y_list şi 20 pentru big_list.

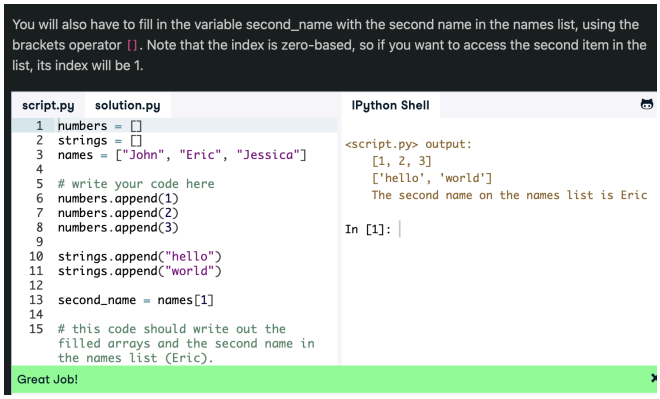


fig.1.13

Pentru a adăuga elemente în liste și a extrage cel de-al doilea element prin indexare (names[1]), s-au utilizat metoda append() și operatorul de indexare [], obținându-se în IPython Shell afișarea listelor [1, 2, 3], ['hello', 'world'] și a numelui Eric.

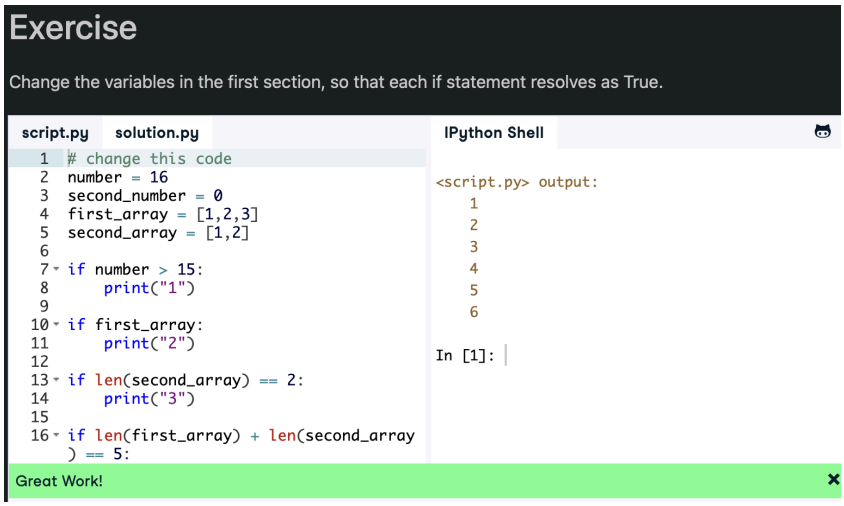


fig.1.14

Pentru a configura corect variabilele și listele astfel încât fiecare instrucțiune condițională să fie validată, s-au folosit operatorii de comparație: >, == și +, alături de funcția len(), obținându-se în IPython Shell afișarea secvențială a numerelor de la 1 la 6 ca urmare a îndeplinirii tuturor condițiilor.

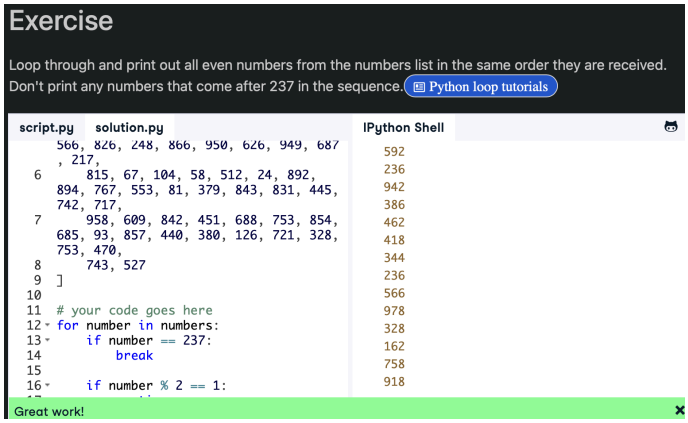


fig.1.15

Pentru a parcurge o listă de numere, a filtra elementele pare și a opri execuția la întâlnirea valorii 237, s-au utilizat o buclă repetitivă for, instrucțiuni condiționale cu operatorul de rest % și comanda break, obținându-se în IPython Shell afișarea secvențială a numerelor pare de dinaintea pragului indicat.



fig.1.16

Pentru a defini funcții reutilizabile și a construi propoziții personalizate, s-au utilizat instrucțiunea def și marcatorul de formatare %s, obținându-se în IPython Shell afișarea secvențială a enunțurilor complete pentru fiecare beneficiu în parte.

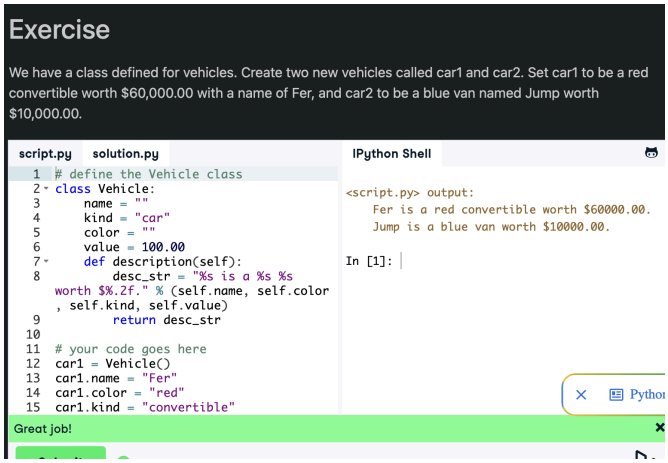


fig.1.17

Pentru a crea obiecte pe baza unei clase definite, a seta atributele specifice acestora și a returna o descriere formatată, s-au utilizat instantierea claselor, operatorul punct (.) și marcatorii de formatare %s și %.2f, obținându-se în IPython Shell afișarea descrierilor detaliate pentru ambele vehicule.

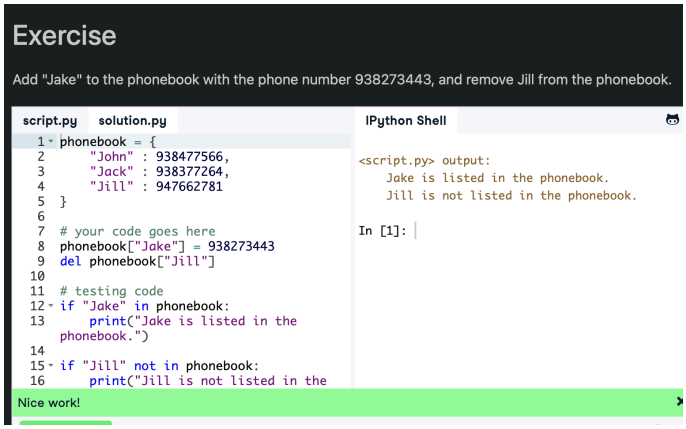


fig.1.18

Pentru a adăuga un contact nou într-un dicționar și a elimina un element existent, s-au utilizat atribuirea pe bază de cheie și instrucțiunea de ștergere del, obținându-se în IPython Shell mesajele de confirmare pentru prezența lui Jake și absența lui Jill.

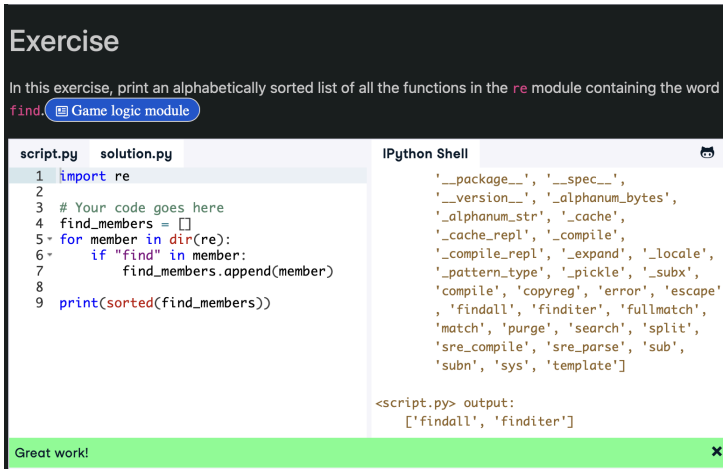


fig.1.19

Pentru a găsi, filtra și sorta alfabetic funcțiile dintr-un modul care conțin un anumit șir de caractere, s-au utilizat importul modulului re, funcția dir(), o buclă for cu o condiție de verificare (if), metoda append() și funcția de sortare sorted(), obținându-se în IPython Shell lista sortată ['findall', 'finditer'].

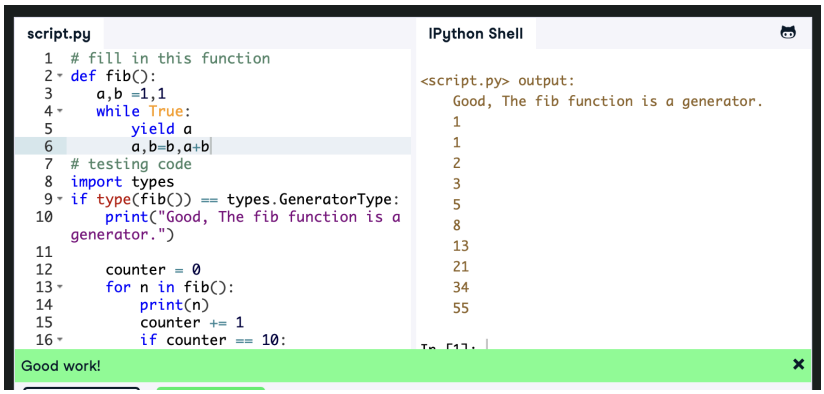


fig.1.20

Pentru a genera o secvență Fibonacci folosind un generator eficient, s-au utilizat instrucțiunea def, bucla while True, cuvântul cheie yield și o buclă for cu un contor de oprire, obținându-se în IPython Shell validarea tipului de generator și afișarea primelor numere din secvență (de la 1 până la 55).

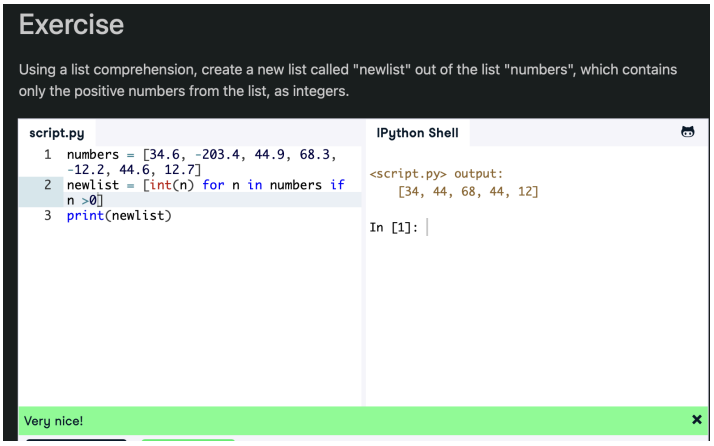


fig.1.21

Pentru a extrage și a converti simultan doar numerele pozitive dintr-o listă de valori zecimale într-o listă de numere întregi, s-a utilizat o expresie inline care aplică funcția de conversie int() pe fiecare element filtrat prin condiția n > 0, obținându-se în IPython Shell lista finală [34, 44, 68, 44, 12].

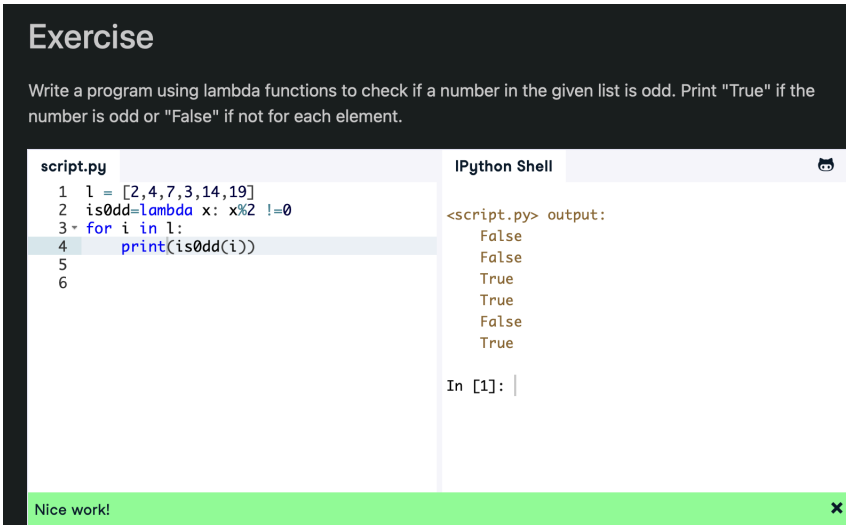


fig.1.22

Pentru a verifica dacă fiecare număr dintr-o listă este impar, s-au utilizat o funcție anonimă de tip lambda care evaluează direct expresia matematică a restului (x % 2 != 0) fără a necesita definirea clasică prin def, împreună cu o buclă for, obținându-se în IPython Shell valorile de adevăr (True sau False) corespunzătoare.

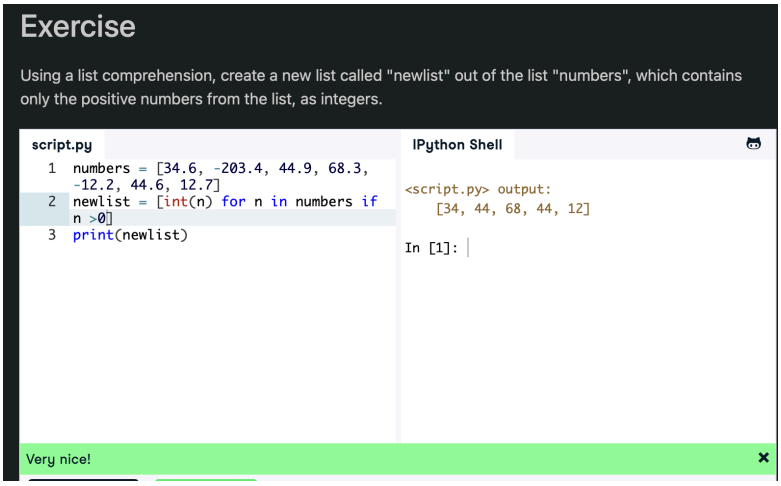


fig.1.23

Pentru a extrage și a converti simultan numerele pozitive, s-a utilizat o expresie inline de tip list comprehension (o construcție compactă care integrează bucla for, filtrarea prin condiția n > 0 și conversia int() într-o singură linie de cod), obținându-se în IPython Shell lista finală [34, 44, 68, 44, 12].

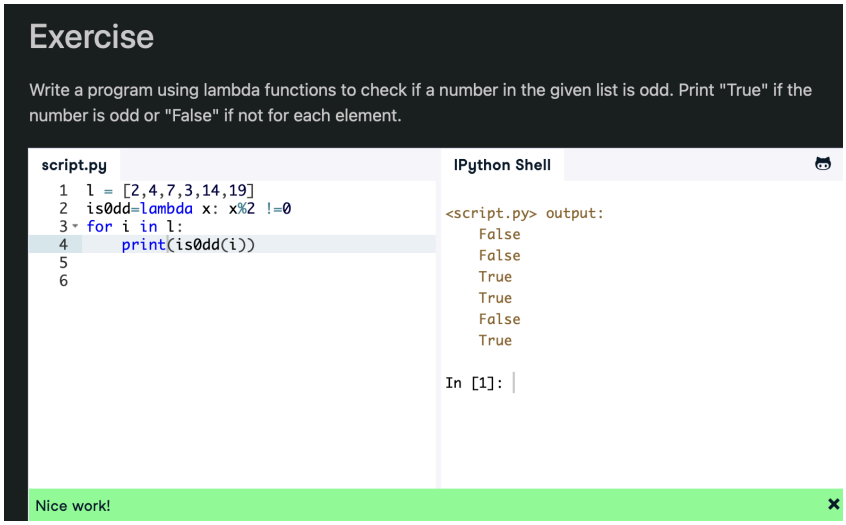


fig.1.24

Pentru a verifica dacă fiecare element dintr-o listă este un număr impar, s-au utilizat o instrucțiune anonimă de tip lambda (o funcție compactă definită inline care evaluează direct expresia matematică a restului prin x % 2 != 0 fără utilizarea clasicului def) împreună cu o buclă for, obținându-se în IPython Shell valorile de adevăr (True sau False) corespunzătoare.

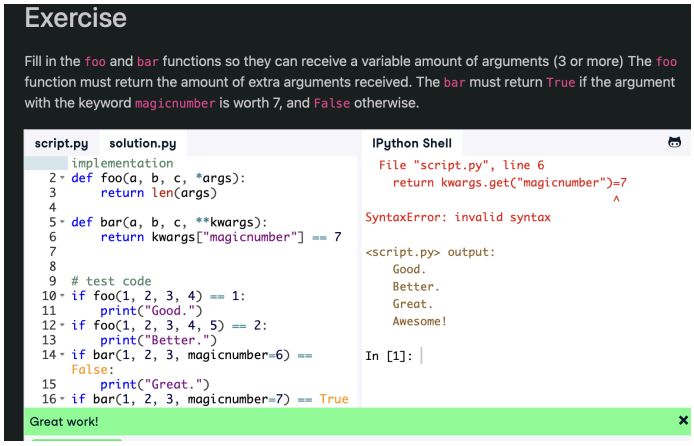


fig.1.25

Pentru a prelua un număr flexibil de argumente suplimentare, s-au utilizat operatorii speciali *args (pentru valori multiple poziționale) și **kwargs (pentru argumente pe bază de cheie și valoare), obținându-se în IPython Shell validarea cu succes a testelor prin mesaje precum „Good” și „Awesome!”.

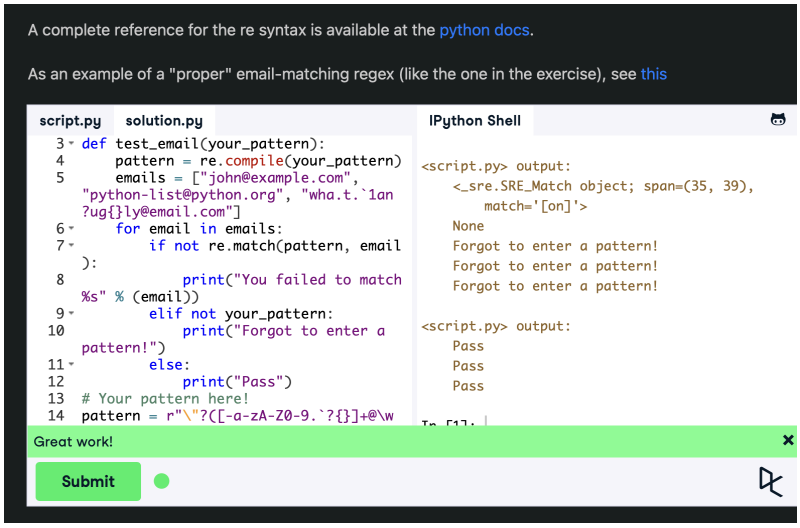


fig.1.26

Pentru a valida un șablon de căutare pentru adrese de e-mail folosind expresii regulate, s-au utilizat funcțiile din modulul re (re.compile și re.match), structuri de control pentru verificare și iterarea printr-o listă de e-mailuri, obținându-se în IPython Shell validarea cu succes a acestora prin afișarea mesajului „Pass”.

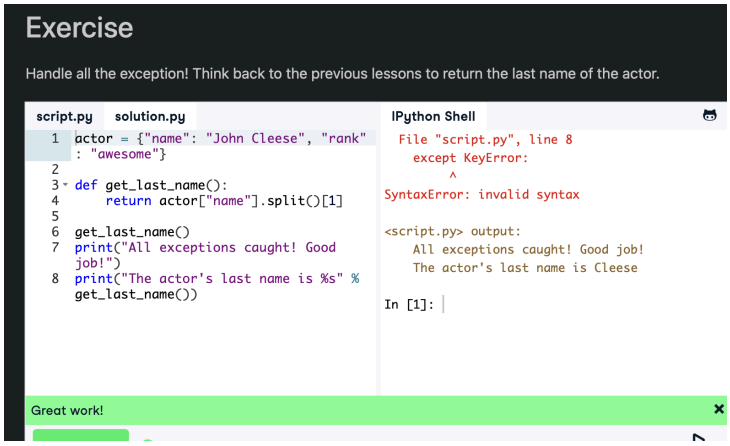


fig.1.27

Pentru a extrage numele de familie dintr-un dicționar, s-au utilizat accesul pe bază de cheie și instrucțiunea de separare a șirurilor prin split(), obținându-se în IPython Shell mesajul de succes și afișarea numelui „Cleese”.



fig.1.28

Pentru a obține participanții unici care au frecventat un singur eveniment, s-au convertit listele de intrare în structuri de tip set și s-a aplicat metoda .symmetric_difference(b), obținându-se în IPython Shell mulțimea exclusivă a acestora {'Eric', 'Jake'}.

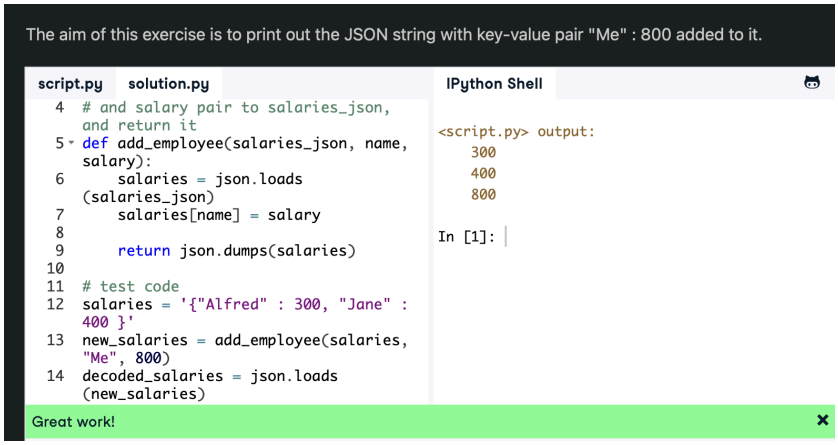


fig.1.29

Pentru a adăuga o nouă pereche cheie-valoare într-un șir JSON, s-au utilizat funcția json.loads() pentru a converti șirul într-un dicționar Python, modificarea directă a elementului și funcția json.dumps() pentru a serializa din nou structura în format JSON, obținându-se în IPython Shell actualizarea corectă a datelor salariale.

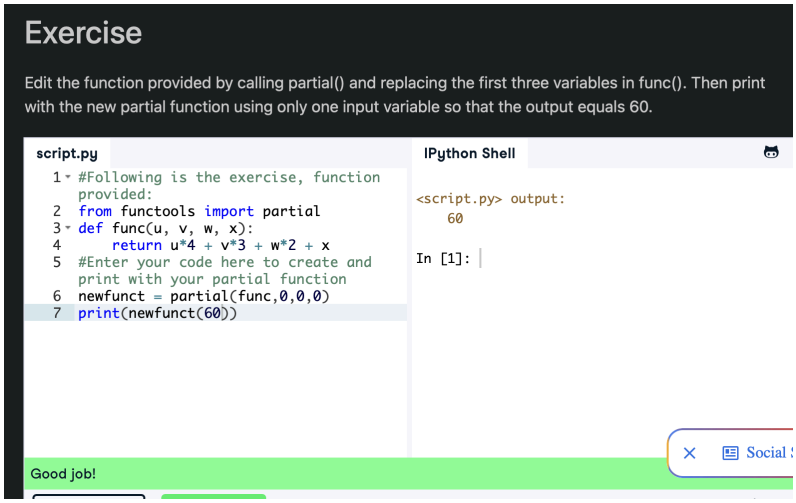


fig.1.30

Pentru a reduce numărul de argumente necesare ale unei funcții, s-a utilizat funcția partial din modulul functools pentru a predefini primele trei variabile ale funcției func cu valoarea zero, permițând apelarea ulterioară doar cu variabila rămasă (newfunct(60)), obținându-se în IPython Shell rezultatul 60.

CONCLUZII

În final , am putut realiza parcurgerea cursurilor de pe **Codecademy** și **LearnPython** , ce reprezinta o experiență foarte utilă, care ajuta să trecem pas cu pas de la bazele limbajului la concepte mult mai avansate.

Prin exercițiile lucrate în ambele platforme, am învățat cum să scriem cod mai curat și mai eficient folosind listări rapide, funcții scurte de tip *lambda*, argumente flexibile(**args,**kwargs)* și instrumente practice pentru texte(*re*), erori și fișiere JSON. Toate aceste noțiuni le-am testat direct în mediu real, oferindu-mi o înțelegere clară și sigură a modului în care funcționează Python în proiecte de zi cu zi.